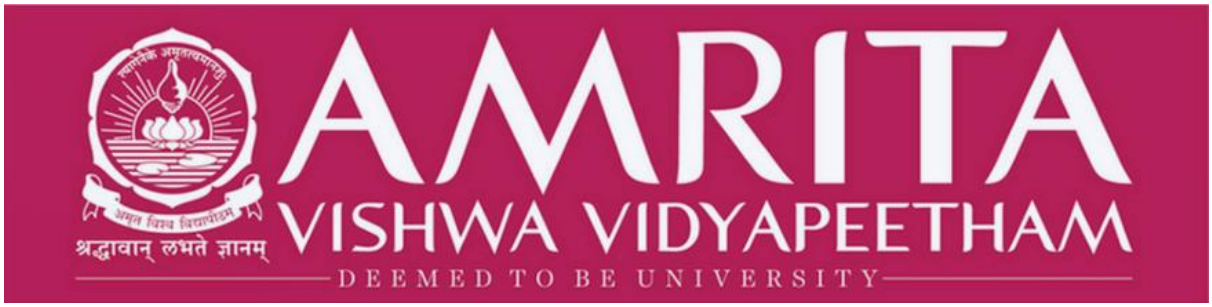


DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

Submitted By	
NAME	T.MANOJ
ROLL NO.	AV.SC.U4CSE252246
YEAR/SEM/SECTION	2nd/3rd/CSE-C
Credits	5

Submitted to	Dr. P. Gayathri
Designation	Asst. Professor (Sl. Grade)
Department	School of Computing

Index :

Sno	Experiments	Page no
1	Recursion programs	
2	Linear and Binary searching	
3	Bubble , Selection , Insertion Sort	
4	Merge and Quick Sort	
5	Singly linked list (SLL)	
6	Doubly linked list (DLL)	
7	Stack operations	
8	Linear and circular queue	
9	BST tree operations	
10	AVL tree operations	
11	Heap tree	
12	Hashing (separate chaining, linear probing , quadratic probing)	

Lab – 1 : Recursion programs

1) AIM: A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed by "Launch!".

Program :

```
def countdown(n):
    if n == 0:
        print("Launch!")
        return
    print(n)
    countdown(n - 1)
n = int(input("Enter countdown number: "))
countdown(n)
```

Output :



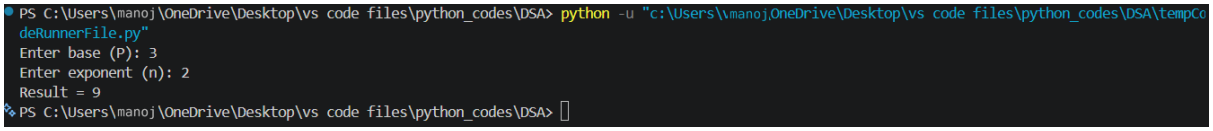
2) AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n (power), where :

P = principal growth factor, n = number of years

Program :

```
def power(P, n):
    if n == 0:
        return 1
    return P * power(P, n - 1)
P = int(input("Enter base (P): "))
n = int(input("Enter exponent (n): "))
print("Result =", power(P, n))
```

Output :



3) AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

Program :

```
def search_employee(emp_list, key, index):
    if index == len(emp_list):
        return False
    if emp_list[index] == key:
        return True
    return search_employee(emp_list, key, index + 1)
employees = [101, 102, 103, 104, 105]
key = int(input("Enter Employee ID to search: "))
if search_employee(employees, key, 0):
    print("Employee ID Found")
else:
    print("Employee ID Not Found")
```

Output :

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\search
.py"
Enter Employee ID to search: 102
Employee ID Found
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA>
```

4) AIM: In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

Program :

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
num = int(input("Enter a number: "))
print("Factorial =", factorial(num))
```

Output :

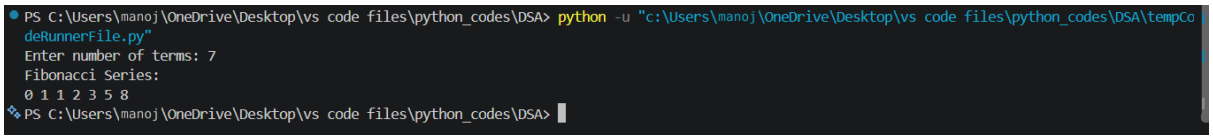
```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\R_factorial.py"
Enter a number: 5
Factorial = 120
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA>
```

5) AIM: Write a recursive Python program to print the first “n” terms of the Fibonacci series.

Program :

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)
terms = int(input("Enter number of terms: "))
print("Fibonacci Series:")
for i in range(terms):
    print(fibonacci(i), end=" ")
```

Output :



Lab – 2 : Linear and Binary searching

1) AIM: Linear search :

Program :

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
arr = []
arr = list(map(int, input("Enter elements of the array separated by space: ").split()))
key = int(input("Enter the element to search for: "))
result = linear_search(arr, key)
if result != -1:
    print(f"Element found at index: {result}")
else:
    print("Element not found in the array.")
```

Output :



2) AIM: Binary search :

Program :

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
arr = list(map(int, input("Enter elements in ascending order:
").split()))
key = int(input("Enter the element to search: "))
result = binary_search(arr, key)
if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found in the array.")
```

Output :



Lab – 3 : Bubble, Selection, Insertion Sort

1) AIM: Bubble sort

Program :

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
arr = list(map(int, input("Enter numbers: ").split()))
print("Sorted List:", bubble_sort(arr))
```

Output :



2) AIM: Selection sort

Program :

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
arr = list(map(int, input("Enter numbers: ").split()))
print("Sorted List:", selection_sort(arr))
```

Output :

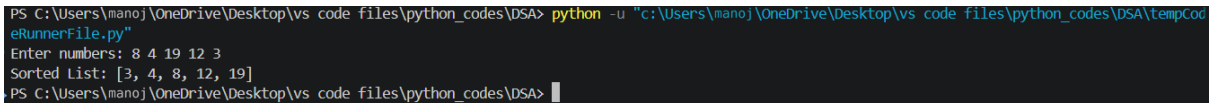
```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\selecti
on_sort.py"
Enter numbers: 20 13 18 2 7
Sorted List: [2, 7, 13, 18, 20]
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA>
```

3) AIM: Insertion Sort

Program :

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
arr = list(map(int, input("Enter numbers: ").split()))
print("Sorted List:", insertion_sort(arr))
```

Output :



Lab – 4 : Merge and Quick Sort

1) Merge sort

Program :

```
def merge(arr, left, mid, right):
    left_part = arr[left:mid + 1]
    right_part = arr[mid + 1:right + 1]
    i = 0
    j = 0
    k = left
    while i < len(left_part) and j < len(right_part):
        if left_part[i] <= right_part[j]:
            arr[k] = left_part[i]
            i += 1
        else:
            arr[k] = right_part[j]
            j += 1
        k += 1
    while i < len(left_part):
        arr[k] = left_part[i]
        i += 1
        k += 1
    while j < len(right_part):
        arr[k] = right_part[j]
        j += 1
        k += 1
def merge_sort(arr, left, right):
    if left < right:
        mid = (left + right) // 2
        merge_sort(arr, left, mid)
        merge_sort(arr, mid + 1, right)
        merge(arr, left, mid, right)
arr = list(map(int, input("Enter numbers: ").split()))
merge_sort(arr, 0, len(arr) - 1)
print("Sorted List:", arr)
```

Output :


PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\tempCod
eRunnerFile.py"
Enter numbers: 8 3 7 12 19 20 22
Sorted List: [3, 7, 8, 12, 19, 20, 22]
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA>

2) Quick Sort

Program :

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return left + middle + right
arr = list(map(int, input("Enter numbers: ").split()))
print("Sorted List:", quick_sort(arr))
```

Output :

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\quick_sort.py"
Enter numbers: 1 2 20 12 17 5 3
Sorted List: [1, 2, 5, 3, 12, 20, 17]
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA>
```

Lab – 5 : Singly linked list (SLL)

1) AIM: Singly linked list (SLL)

Program :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter the number of the nodes: "))
        for i in range(n):
            data = int(input("enter the data for node : "))
            new_node = Node(data)
            if self.head is None:
                self.head = new_node
            else:
                current = self.head
                while current.next:
                    current = current.next
                current.next = new_node

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

    def count(self):
        count = 0
        current = self.head
        while current:
```

```
            count += 1
            current = current.next
        print(f"Total number of nodes: {count}")

    def insert_at_beginning(self,data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_end(self,data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node

    def insert_at_index(self, index, data):
        if index < 0:
            print("Index cannot be negative.")
            return
        new_node = Node(data)
        if index == 0:
            new_node.next = self.head
            self.head = new_node
            return
        current = self.head
        for i in range(index - 1):
            if current is None:
                print("Index out of bounds.")
                return
            current = current.next
        if current is None:
            print("Index out of bounds.")
```

```
            return
        new_node.next = current.next
        current.next = new_node

    def delete_at_beginning(self):
        if self.head is None:
            print("List is empty. Cannot delete.")
            return
        self.head = self.head.next

    def delete_at_end(self):
        if self.head is None:
            print("List is empty. Cannot delete.")
            return
        if self.head.next is None:
            self.head = None
            return
        current = self.head
        while current.next.next:
            current = current.next
        current.next = None

    def delete_at_index(self, index):
        if index < 0:
            print("Index cannot be negative.")
            return
        if self.head is None:
            print("List is empty. Cannot delete.")
            return
        if index == 0:
            self.head = self.head.next
            return
        current = self.head
        for i in range(index - 1):
            if current is None or current.next is None:
                print("Index out of bounds.")
                return
```

```
            current = current.next
        if current.next is None:
            print("Index out of bounds.")
            return
        current.next = current.next.next

    def delete(self, data):
        if self.head is None:
            return
        if self.head.data == data:
            self.head = self.head.next
            return
        current = self.head
        while current.next is not None:
            if current.next.data == data:
                current.next = current.next.next
                return
            current = current.next

    def searching(self, data):
        current = self.head
        index = 0
        while current:
            if current.data == data:
                print(f"Data {data} found at index {index}.")
                return
            current = current.next
            index += 1
        print(f"Data {data} not found in the list.")

    def sorting(self):
        if self.head is None:
            return
        current = self.head
        while current:
            index = current.next
            while index:
```

```
            if current.data > index.data:
                current.data, index.data = index.data, current.data
            index = index.next
        current = current.next

def main():
    linked_list = LinkedList()
    while True:
        print("\nMenu:")
        print("1. Create Linked List")
        print("2. Display Linked List")
        print("3. Insert at Beginning")
        print("4. Insert at End")
        print("5. Insert at Index")
        print("6. Delete at Beginning")
        print("7. Delete at End")
        print("8. Delete at Index")
        print("9. Search for an Element")
        print("10. Count Nodes")
        print("11. Delete a specific element")
        print("12. Sort the linked list")
        print("13. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            linked_list.create()
        elif choice == 2:
            linked_list.display()
        elif choice == 3:
            data = int(input("Enter data to insert at beginning: "))
            linked_list.insert_at_beginning(data)
        elif choice == 4:
            data = int(input("Enter data to insert at end: "))
            linked_list.insert_at_end(data)
        elif choice == 5:
            index = int(input("Enter index to insert at: "))
```

```
            data = int(input("Enter data to insert: "))
            linked_list.insert_at_index(index, data)
        elif choice == 6:
            linked_list.delete_at_beginning()
        elif choice == 7:
            linked_list.delete_at_end()
        elif choice == 8:
            index = int(input("Enter index to delete at: "))
            linked_list.delete_at_index(index)
        elif choice == 9:
            data = int(input("Enter data to search for: "))
            linked_list.searching(data)
        elif choice == 10:
            linked_list.count()
        elif choice == 11:
            data = int(input("Enter data to delete: "))
            linked_list.delete(data)
        elif choice == 12:
            linked_list.sorting()
        elif choice == 13:
            print("Exiting...")
            break
        else:
            print("Invalid choice. Please try again.")

main()
```

Output :

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\tempCodeRunnerFile.py"
Menu:
1. Create Linked List
2. Display Linked List
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Search for an Element
10. Count Nodes
11. Delete a specific element
12. Sort the linked list
13. Exit
Enter your choice: 1
Enter the number of the nodes: 5
enter the data for node : 1
enter the data for node : 2
enter the data for node : 3
enter the data for node : 4
enter the data for node : 5

Menu:
1. Create Linked List
2. Display Linked List
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Search for an Element
10. Count Nodes
11. Delete a specific element
12. Sort the linked list
13. Exit
Enter your choice: 2
1 -> 2 -> 3 -> 4 -> 5 -> None
```

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\tempCodeRunnerFile.py"
Menu:
1. Create Linked List
2. Display Linked List
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Search for an Element
10. Count Nodes
11. Delete a specific element
12. Sort the linked list
13. Exit
Enter your choice: 6

Menu:
1. Create Linked List
2. Display Linked List
3. Insert at Beginning
4. Insert at End
5. Insert at Index
6. Delete at Beginning
7. Delete at End
8. Delete at Index
9. Search for an Element
10. Count Nodes
11. Delete a specific element
12. Sort the linked list
13. Exit
Enter your choice: 2
2 -> 3 -> 4 -> 5 -> None
```

Lab – 6 : Doubly linked list (DLL)

1) AIM: Doubly linked list (DLL)

Program :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)
            if self.head is None:
                self.head = self.tail = new_node
            else:
                new_node.prev = self.tail
                self.tail.next = new_node
                self.tail = new_node

    def insert_at_beginning(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = self.tail = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node

    def insert_at_end(self, data):
```

```
        new_node = Node(data)
        if self.tail is None:
            self.head = self.tail = new_node
        else:
            new_node.prev = self.tail
            self.tail.next = new_node
            self.tail = new_node

    def insert_at_index(self, index, data):
        if index < 0:
            print("Invalid index")
            return
        if index == 0:
            self.insert_at_beginning(data)
            return
        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid index")
            return
        if temp.next is None:
            self.insert_at_end(data)
            return
        new_node = Node(data)
        new_node.next = temp.next
        new_node.prev = temp
        temp.next.prev = new_node
        temp.next = new_node

    def delete_at_index(self, index):
        if self.head is None or index < 0:
            print("Invalid index")
            return
```

```
        temp = self.head
        for i in range(index):
            if temp is None:
                print("Invalid index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid index")
            return
        if temp.prev is None:
            self.head = temp.next
        else:
            temp.prev.next = temp.next
        if temp.next is None:
            self.tail = temp.prev
        else:
            temp.next.prev = temp.prev

    def delete_value(self, data):
        temp = self.head
        index = 0
        while temp is not None and temp.data != data:
            temp = temp.next
            index += 1
        if temp is None:
            print("Value not found")
            return
        self.delete_at_index(index)

    def count(self):
        count = 0
        temp = self.head
        while temp is not None:
            count += 1
            temp = temp.next
        print("Total number of nodes:", count)
```

```
    def display_forward(self):
        temp = self.head
        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")

    def display_backward(self):
        temp = self.tail
        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.prev
        print("None")

def main():
    dll = DoublyLinkedList()
    while True:
        print("\n1.Create  2.Insert beginning  3.Insert end  4.Insert
index")
        print("5.Delete value  6.Delete index  7.Count  8.Display
forward")
        print("9.Display backward  10.Exit")
        choice = int(input("Enter choice: "))
        if choice == 1:
            dll.create()
        elif choice == 2:
            dll.insert_at_beginning(int(input("Enter data: ")))
        elif choice == 3:
            dll.insert_at_end(int(input("Enter data: ")))
        elif choice == 4:
            dll.insert_at_index(int(input("Enter index: ")), int(input("Enter
data: ")))
        elif choice == 5:
            dll.delete_value(int(input("Enter value: ")))
        elif choice == 6:
            dll.delete_at_index(int(input("Enter index: ")))
        elif choice == 7:
```

```
            dll.count()
        elif choice == 8:
            dll.display_forward()
        elif choice == 9:
            dll.display_backward()
        elif choice == 10:
            break
        else:
            print("Invalid choice")

main()
```

Output :

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\tempCodeRunnerFile.py"

1.Create  2.Insert beginning  3.Insert end  4.Insert index
5.Delete value  6.Delete index  7.Count  8.Display forward
9.Display backward  10.Exit
Enter choice: 1
Enter number of nodes: 6
Enter data: 1
Enter data: 2
Enter data: 3
Enter data: 4
Enter data: 5
Enter data: 6

1.Create  2.Insert beginning  3.Insert end  4.Insert index
5.Delete value  6.Delete index  7.Count  8.Display forward
9.Display backward  10.Exit
Enter choice: 8
1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <-> None

1.Create  2.Insert beginning  3.Insert end  4.Insert index
5.Delete value  6.Delete index  7.Count  8.Display forward
9.Display backward  10.Exit
Enter choice: 5
Enter value: 3

1.Create  2.Insert beginning  3.Insert end  4.Insert index
5.Delete value  6.Delete index  7.Count  8.Display forward
9.Display backward  10.Exit
Enter choice:
```

LAB-7:

Lab – 7 : Stack operations

1) AIM: Stack operations using arrays :

Program :

```
class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
```

```
            for i in range(self.top, -1, -1):
                print(self.stack[i])

size = int(input("Enter stack size: "))
s = Stack(size)
while True:
    print("\n1.Push  2.Pop  3.Peek  4.Display  5.Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        s.push(int(input("Enter item: ")))
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

2) AIM: Stack operations using linked list :

Program :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed into stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            item = self.top.data
            self.top = self.top.next
            print(item, "popped from stack")

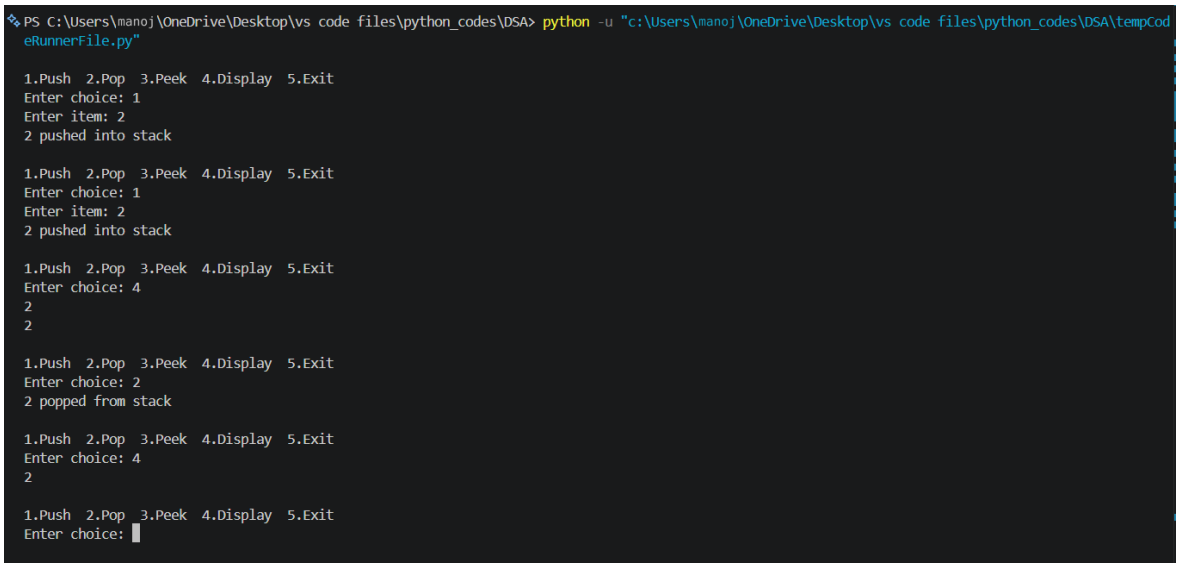
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
            return
        temp = self.top
        while temp is not None:
```

```
            print(temp.data)
            temp = temp.next

s = Stack()
while True:
    print("\n1.Push  2.Pop  3.Peek  4.Display  5.Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        s.push(int(input("Enter item: ")))
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

Output :



LAB-8:

Lab – 8 : Linear and circular queue

1) AIM: Implement enqueue, dequeue, peek, and display using an array.

```
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, data):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear += 1
            self.queue[self.rear] = data
            print(data, "inserted into queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        else:
            data = self.queue[self.front]
            self.queue[self.front] = None
            if self.front == self.rear:
                self.front = self.rear = -1
            else:
                self.front += 1
            print(data, "deleted from queue")

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
```

```
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print(self.queue[self.front:self.rear + 1])

size = int(input("Enter queue size: "))
q = Queue(size)
while True:
    print("\n1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        q.enqueue(int(input("Enter item: ")))
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

2) Implement enqueue, dequeue, peek, and display using an linked list.

Program :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)
        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node
        print(data, "inserted into queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            data = self.front.data
            self.front = self.front.next
            if self.front is None:
                self.rear = None
            print(data, "deleted from queue")

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)
```

```
    def display(self):
        if self.front is None:
            print("Queue is empty")
            return
        temp = self.front
        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

q = Queue()
while True:
    print("\n1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        q.enqueue(int(input("Enter item: ")))
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

Output :

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\tempCod
eRunnerFile.py"
Enter queue size: 4

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 1
1 inserted into queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 2
2 inserted into queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 3
3 inserted into queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 4
[1, 2, 3]

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 2
1 deleted from queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 4
[2, 3]

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice:
```

3) Implement circular queue enqueue, dequeue, and display using an array.

Program :

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into the queue")


    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        # Only one element is present
        elif self.front == self.rear:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
            print(item, "deleted from the queue")
        # More than one element
        else:
            item = self.queue[self.front]
```

```
            self.queue[self.front] = None
            self.front = (self.front + 1) % self.size
            print(item, "deleted from the queue")

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size

# Create Circular Queue
size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)
while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.display()
    elif choice == 4:
        print("Program terminated.")
        break
```

```
    else:
        print("Invalid choice")
```

4) Implement circular queue enqueue, dequeue, peek, and display using a circular linked list.

Program :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)
        if self.front is None:
            self.front = self.rear = new_node
            new_node.next = new_node
        else:
            new_node.next = self.front
            self.rear.next = new_node
            self.rear = new_node
        print(data, "inserted into the queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        elif self.front == self.rear:
            data = self.front.data
            self.front = self.rear = None
            print(data, "deleted from the queue")
        else:
            data = self.front.data
            self.front = self.front.next
            self.rear.next = self.front
            print(data, "deleted from the queue")
```

```
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
            return
        temp = self.front
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.front:
                break
        print("(front)")

q = CircularQueue()
while True:
    print("\n1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        q.enqueue(int(input("Enter item: ")))
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

Output :

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\tempCodeRunnerFile.py"

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 2
2 inserted into the queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 3
3 inserted into the queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 1
Enter item: 4
4 inserted into the queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 4
2 -> 3 -> 4 -> (front)

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 2
2 deleted from the queue

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice: 4
3 -> 4 -> (front)

1.Enqueue  2.Dequeue  3.Peek  4.Display  5.Exit
Enter choice:
```

LAB-9:

Lab – 9 : BST tree operations

1) AIM: BST tree operations

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
```

```
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
```

```
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1


def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
```

```
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)


def level_order(root):
    h = height(root)
    for level in range(h):
        print_level(root, level)


#
================================================================
===
# MAIN PROGRAM
#
================================================================
===

root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
    print("6. FIND MAXIMUM")
```

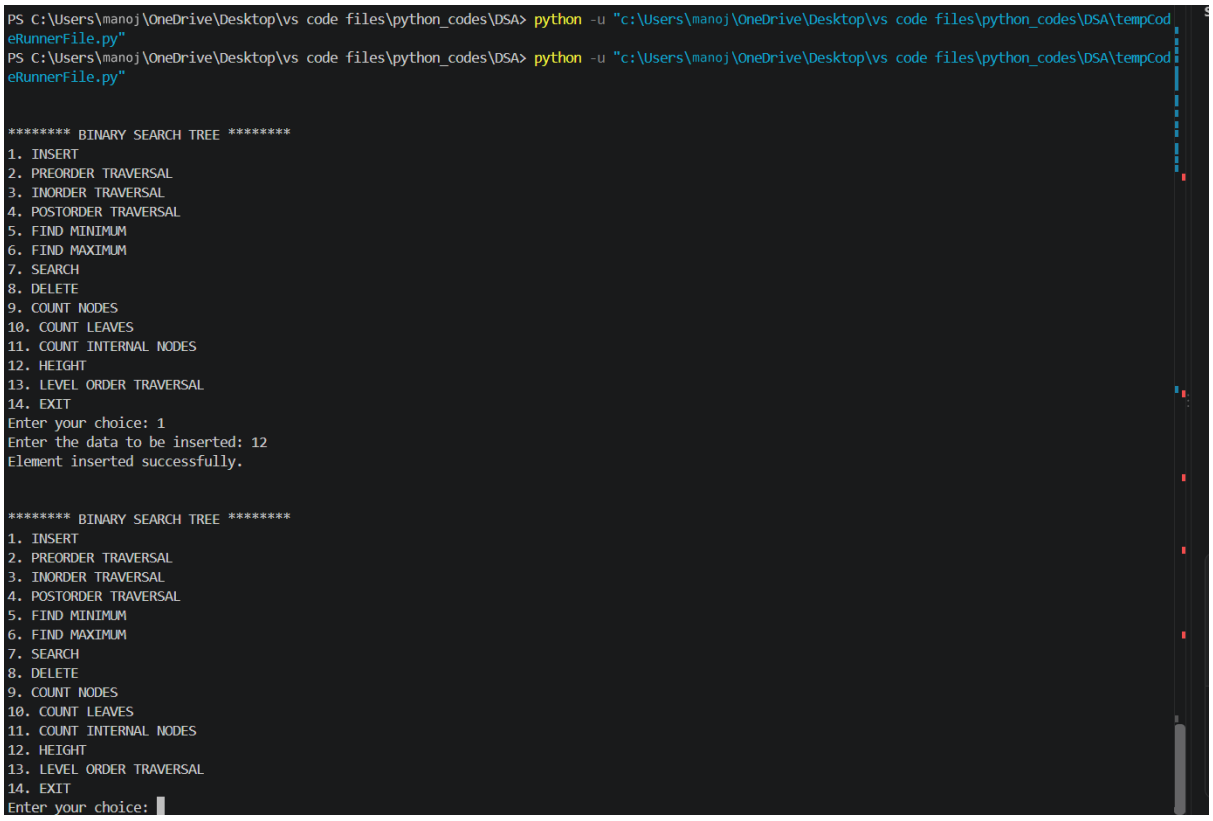
```
print("7. SEARCH")
print("8. DELETE")
print("9. COUNT NODES")
print("10. COUNT LEAVES")
print("11. COUNT INTERNAL NODES")
print("12. HEIGHT")
print("13. LEVEL ORDER TRAVERSAL")
print("14. EXIT")

choice = int(input("Enter your choice: "))
if choice == 1:
    data = int(input("Enter the data to be inserted: "))
    root = insert(root, data)
    print("Element inserted successfully.")
elif choice == 2:
    print("Preorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
        preorder(root)
        print()
elif choice == 3:
    print("Inorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
        inorder(root)
        print()
elif choice == 4:
    print("Postorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
        postorder(root)
        print()
elif choice == 5:
    node = find_min(root)
```

```
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 9:
        print("Number of nodes:", count_nodes(root))
    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
```

```
        break
    else:
        print("Invalid choice. Please try again.")
```

Output :



LAB-10:

Lab – 10 : AVL tree operations

1) AVL tree operations

```
class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0

#AVL TREE Program
def height(root):
    if root is None:
        return -1
    else:
        return root.h

# LL Rotation
def LL(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# RR Rotation
def RR(x):
    y = x.right
    x.right = y.left
    y.left = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# LR Rotation
def LR(k3):
    k3.left = RR(k3.left)
```

```
    return LL(k3)

# RL Rotation
def RL(k3):
    k3.right = LL(k3.right)
    return RR(k3)

# Inorder Traversal
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


# Preorder Traversal
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


# Postorder Traversal
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

# Insert a node into AVL Tree
def insert(root, x):
    if root is None:
        return Node(x)

    if x < root.data:
        root.left = insert(root.left, x)
        if height(root.left) - height(root.right) == 2:
```

```
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    elif x > root.data:
        root.right = insert(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
        print("Duplicates not allowed")
    root.h = max(height(root.left),height(root.right)) + 1
    return root


# Delete a node from AVL Tree
def delete(root, x):
    # Element not found
    if root is None:
        return None

    # Search in left subtree
    if x < root.data:
        root.left = delete(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    # Search in right subtree
    elif x > root.data:
        root.right = delete(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
```

```
            root = RR(root)
        else:
            root = RL(root)

    # When target found
    else:
        if root.left is None and root.right is None:
            root = None
        elif root.right is None:
            root = root.left
        elif root.left is None:
            root = root.right
        else:
            successor = root.right   # Find inorder successor
            while successor.left is not None:
                successor = successor.left
            root.data = successor.data   #Copy successor data
            root.right = delete(root.right, successor.data)     # Delete the
inorder successor

        root.h = max(height(root.left), height(root.right)) + 1
    return root

# ----------------------------
# Main Program
# ----------------------------

root = None

while True:

    print("\n***** AVL TREE MENU *****")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. DELETE")
```

```
print("6. EXIT")

choice = int(input("Enter your choice: "))
if choice == 1:
    data = int(input("Enter the data to be inserted: "))
    root = insert(root, data)
elif choice == 2:
    print("Preorder Traversal:")
    preorder(root)
    print()
elif choice == 3:
    print("Inorder Traversal:")
    inorder(root)
    print()
elif choice == 4:
    print("Postorder Traversal:")
    postorder(root)
    print()
elif choice == 5:
    data = int(input("Enter the data to be deleted: "))
    root = delete(root, data)
elif choice == 6:
    print("Program terminated.")
    break
else:
    print("Invalid choice!")
```

Output :

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\tempCod
eRunnerFile.py"
***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 1
Enter the data to be inserted: 12

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 1
Enter the data to be inserted: 12
Duplicates not allowed

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 2
Preorder Traversal:
12

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
```

LAB-10:

Lab – 11 : Heap tree

1) AIM: Heap tree

Program :

```
def insert(heap, value):
    heap.append(value)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] > heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

def delete(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()

    n = len(heap)
    i = 0
    # Move the root downward
    while True:
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and heap[left] > heap[largest]:
            largest = left
        if right < n and heap[right] > heap[largest]:
            largest = right
        if largest != i:
            heap[i], heap[largest] = heap[largest], heap[i]
```

```
            i = largest
        else:
            break

def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
        print("Heap elements:", heap)

#Main Program
heap = []
while True:
    print("\n--- MAX HEAP MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        value = int(input("Enter value to insert: "))
        insert(heap, value)
    elif choice == 2:
        delete(heap)
    elif choice == 3:
        display(heap)
    elif choice == 4:
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

Program :

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\tempCod
eRunnerFile.py"

--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 12

--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 11

--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Heap elements: [12, 11]

--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice:
```

LAB-12:

Lab – 12 : Hashing (separate chaining, linear probing, quadratic probing)

1) Resolve collisions using separate chaining.

Program :

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

SIZE = 10
ht = [None] * SIZE


def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode


def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")
```

```python
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

Output :

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\tempCodeRunnerFile.py"

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 12

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 11

1. Insert
2. Display
3. Exit
Enter your choice: 2

Hash table values:
0: None
1: 11 -> None
2: 12 -> None
3: None
4: None
5: None
6: None
7: None
8: None
9: None

1. Insert
2. Display
3. Exit
Enter your choice:
```

2) Resolve collisions using linear probing.

Program :

```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")


def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")

def search(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            print("Element found at index", index)
            return
        if a[index] == -1:
            break
    print("Element not found")


def displayValue():
    print("Index\tValue")
    for i in range(SIZE):
        print(i, "\t", a[i])
```

```
# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Delete")
    print("4. Search")
    print("5. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("Values are:")
        displayValue()
    elif ch == 3:
        value = int(input("Enter value to delete: "))
        deleteValue(value)
    elif ch == 4:
        value = int(input("Enter value to search: "))
        search(value)
    elif ch == 5:
        repeat = False
    else:
        print("Invalid choice")
```

Output :

```
PS C:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA> python -u "c:\Users\manoj\OneDrive\Desktop\vs code files\python_codes\DSA\tempCod
eRunnerFile.py"

1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 1
Enter value: 12
Element inserted

1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 1
Enter value: 11
Element inserted

1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 2
Values are:
Index   Value
0       -1
1       11
2       12
3       -1
4       -1
5       -1
6       -1
7       -1
8       -1
9       -1
```

3) Resolve collisions using quadratic probing.

Program :

```
SIZE = 10
a = [-1] * SIZE

def insertValue(value):
    for i in range(SIZE):
        index = ((value % SIZE) + i * i) % SIZE
    if a[index] == -1:
            a[index] = value
            print("Element inserted at index", index)
            return
    print("No empty slot found in the probe sequence")


def displayValue():
    print("Values are:")
    for i in range(SIZE):
        print(a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        displayValue()
    elif ch == 3:
        repeat = False
    else:
```

```
        print("Invalid choice")
```

Output :

